

CI/CD Pipeline Setup Guide

How to go from an empty AWS account to the IT Support Chatbot running on EKS.

The pipeline is four workflows. The first three are run in order to deploy, and the fourth is run when you want to tear everything down. Each is manually triggered (`workflow_dispatch`) — nothing deploys on a git push.

#	Workflow	When you run it
1	03-tf-bootstrap.yaml	Once, ever
2	04-infra-eks.yaml	When cluster infra changes
3	05-build-deploy.yaml	Every app release
4	06-destroy.yaml	When you want to delete the app and EKS cluster

```
03-tf-bootstrap → S3 state bucket + DynamoDB lock table
                  |
                  ▼
04-infra-eks → plan —[approval]→ apply → EKS cluster
                                      |
                                      ▼
05-build-deploy → build → Docker Hub —[approval]→ deploy
06-destroy → app teardown → destroy plan —[approval]→ destroy
```

Prerequisites

- An AWS account with permission to create IAM users, VPCs, and EKS clusters
- A Docker Hub account
- Admin access to this GitHub repository

Cost warning. An EKS control plane bills roughly **\$0.10/hour (~\$72/month)** and keeps billing whether or not anything is deployed. On top of that: one `t3.small` worker (~\$15/month) and a Classic Load Balancer (~\$18/month) if you use `service.yaml`. NAT is avoided by design — see [Step 5](#). Budget **~\$105/month** if you leave everything running. When done, run [06-destroy.yaml](#).

Terraform lock note. The infra and destroy workflows now share a GitHub Actions concurrency group and wait up to 10 minutes for the DynamoDB lock on `eks/terraform.tfstate`. If a run still fails with `ConditionalCheckFailedException`, another job is actively holding the lock or a stale lock item was left behind. Cancel any in-flight Terraform run first. Only if you have confirmed no apply is still running should you clear the lock with `terraform force-unlock <LOCK_ID>` from `terraform/03-eks/` after `terraform init` against the same backend.

Step 1 — Create the AWS IAM user

The pipeline uses static access keys (chosen for simplicity in this teaching repo). Create a dedicated user — do not reuse your console login.

1. AWS Console → **IAM** → **Users** → **Create user**
2. Name it `github-actions-ness-itbot`. Do **not** enable console access.
3. Attach permissions. For a learning environment, these AWS-managed policies work:
 - `AmazonEKSClusterPolicy`
 - `AmazonEC2FullAccess` (Terraform builds the VPC, subnets, node groups)
 - `IAMFullAccess` (EKS requires creating service-linked roles)
 - `AmazonS3FullAccess` and `AmazonDynamoDBFullAccess` (Terraform state)
4. After creating the user: **Security credentials** → **Create access key** → choose *Third-party service*. Copy both values now; the secret is shown once.

`IAMFullAccess` and `*FullAccess` are broader than production should ever use. For a real deployment, scope these down to the specific resources Terraform manages.

Step 2 — Create the Docker Hub token

1. `hub.docker.com` → **Account Settings** → **Personal access tokens**
2. **Generate new token**, description `github-actions`, permissions **Read & Write**
3. Copy the token — it is shown only once

Use the token, never your account password.

Step 3 — Add GitHub secrets

Repo → **Settings** → **Secrets and variables** → **Actions** → **New repository secret**.

Secret name	Value
<code>AWS_ACCESS_KEY_ID</code>	From Step 1
<code>AWS_SECRET_ACCESS_KEY</code>	From Step 1
<code>DOCKERHUB_USERNAME</code>	Your Docker Hub username
<code>DOCKERHUB_TOKEN</code>	From Step 2

Names must match exactly — the workflows reference them verbatim.

Step 4 — Create the `production` environment

This is what makes `terraform apply` and the EKS deploy pause for human approval instead of running unattended.

1. Repo → **Settings** → **Environments** → **New environment**
2. Name it exactly `production`
3. Tick **Required reviewers**, add yourself, and save

Skip this and both gated jobs still run — just without the approval pause.

Step 5 — Terraform and k8s layout

Both directories are already in the repo and wired to the pipeline. This section explains what CI drives and what it deliberately ignores.

`terraform/03-eks/`

The only Terraform directory; `TF_DIR` in `04-infra-eks.yaml` points at it.

File	Creates
<code>0-provider.tf</code>	AWS + TLS providers, S3 backend stub
<code>1-vpc.tf</code> / <code>2-igw.tf</code>	VPC and internet gateway
<code>3-subnets.tf</code>	Two public + two private subnets across 2 AZs
<code>4-nat.tf</code>	NAT gateway — disabled by default , see below
<code>5-routes.tf</code>	Route tables and subnet associations
<code>6-eks.tf</code>	EKS cluster and its IAM role
<code>7-nodes.tf</code>	Managed node group in the public subnets
<code>8-iam-oidc.tf</code>	OIDC provider — prerequisite for IRSA
<code>variables.tf</code> / <code>outputs.tf</code>	Inputs and cluster outputs

`4-nat.tf` must stay even though `enable_nat_gateway` is `false`. `5-routes.tf` references `aws_nat_gateway.nat[0]` inside a `dynamic` block, and Terraform resolves that reference at parse time regardless of the count — deleting the file breaks `terraform validate`. The private subnets are load-bearing for the same reason.

`0-provider.tf` carries an intentionally empty backend block — the workflow injects bucket, key, region, and lock table via `-backend-config` flags at init time:

```
backend "s3" {}
```

Defaults relevant to the pipeline, all in `variables.tf`:

Variable	Default	Note
<code>cluster_name</code>	<code>ness-itbot-eks</code>	Must match <code>EKS_CLUSTER</code> in workflow 05
<code>kubernetes_version</code>	<code>1.31</code>	Keep kubectl in workflow 05 within one minor
<code>create_vpc</code>	<code>true</code>	Builds its own VPC
<code>enable_nat_gateway</code>	<code>false</code>	Not needed — see below

Variable	Default	Note
<code>node_desired_size</code>	1	One <code>t3.small</code> worker

Two version pins must stay in step: `kubernetes_version` here and the `azure/setup-kubectl` version in `05-build-deploy.yaml`. `kubectl` supports one minor version of skew either way; drift further and `kubectl apply` starts failing on unrecognised API fields.

Worker nodes run in the public subnets (`7-nodes.tf`). They get public IPs and reach the EKS API and image registries through the internet gateway, so no NAT gateway is required — saving roughly \$32/month. The trade-off is that nodes are internet-facing. For a production posture, move the node group back to the private subnets and set `enable_nat_gateway = true`.

k8s/

Every manifest here is applied by `kubectl apply -f k8s/`:

File	What it is
<code>deployment.yaml</code>	Chatbot Deployment
<code>service.yaml</code>	LoadBalancer on port 80 → 8501
<code>nodeportsvc.yaml</code>	NodePort 31446 alternative

Names are already aligned with the workflow `env:` block:

Workflow variable	Value	Meaning
<code>K8S_DEPLOYMENT</code>	<code>it-support-chatbot</code>	Deployment <code>metadata.name</code>
<code>K8S_CONTAINER</code>	<code>chatbot</code>	Container name targeted by <code>kubectl set image</code>
<code>K8S_NAMESPACE</code>	<code>it-support</code>	Created automatically if absent

The Deployment reads AWS credentials from the `aws-credentials` secret, which the pipeline recreates on every run:

```
envFrom:
  - secretRef:
      name: aws-credentials
```

That secret supplies `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, and `AWS_DEFAULT_REGION` — exactly the three variables `config.py` validates at startup.

Replicas are fixed at 1. Each pod builds its own in-memory Chroma index and keeps chat history in an `InMemorySaver`, so replicas would not share state and users would get inconsistent sessions. Scaling out requires an external vector store and a shared checkpoint first.

The `image:` in `deployment.yaml` is a placeholder token — CI renders it to the exact SHA-tagged image built by that run before `kubectl apply`.

Step 6 — Verify the image locally

The [dockerfile](#) is ready to build. It runs Streamlit bound to `0.0.0.0:8501` as a non-root user on `python:3.11-slim`, matching the Python version CI uses.

A [.dockerignore](#) keeps `.env`, `chroma_db/`, `terraform/`, `.github/`, and `docs/` out of the build context. **This matters for more than image size:** the image is pushed to a public Docker Hub repo, and a file copied into a layer stays in the image history even if a later layer deletes it. A committed `.env` would be permanently extractable by anyone who pulls it.

Build and run before pushing:

```
docker build -t itbot-test -f dockerfile .
docker run --rm -p 8501:8501 --env-file .env itbot-test
```

Open <http://localhost:8501>. First launch takes a minute or two — the container embeds `it_sector.txt` through Bedrock to build its Chroma index, which is why the Deployment sets a generous `startupProbe`.

Check the health endpoint the k8s probes use:

```
curl -f http://localhost:8501/_stcore/health # expects: ok
```

The container runs as the non-root `appuser`, which owns `/app`. If you add a volume mount or write outside `/app`, adjust ownership or the write will fail with a permission error.

Step 7 — Run the bootstrap (once)

Actions → TF Bootstrap (run once) → Run workflow. Type `bootstrap` in the confirmation box.

Creates:

- S3 bucket `ness-itbot-tfstate` — versioned, AES256-encrypted, public access blocked
- DynamoDB table `ness-itbot-tflock` — prevents concurrent Terraform runs corrupting state

S3 bucket names are globally unique across all AWS accounts. If creation fails with `BucketAlreadyExists`, edit `STATE_BUCKET` in both `03-tf-bootstrap.yaml` and `04-infra-eks.yaml` — append your AWS account ID, e.g. `ness-itbot-tfstate-123456789012`. The two files must always agree.

The job is idempotent: re-running it detects existing resources and skips them.

Step 8 — Provision the EKS cluster

Actions → Infra - EKS (Terraform) → Run workflow.

First run with action: `plan`. Read the plan in the job summary and confirm it creates what you expect. Nothing is changed by a plan.

Then run with action: `apply`. The `apply` job waits for your approval in the `production` environment. The plan is passed from the plan job as an artifact, so `apply` executes exactly what you reviewed — no drift between the two.

Expect **15–20 minutes** for a first cluster.

Step 9 — Build and deploy

Actions → Build & Deploy to EKS → Run workflow.

Input	Meaning
<code>image_tag</code>	Leave blank to tag with the short commit SHA (recommended)
<code>deploy</code>	Untick to build and push without deploying

The build job pushes two tags: the immutable SHA tag and `:latest`. **Only the SHA tag is deployed** — deploying `:latest` makes rollbacks ambiguous because the tag moves.

The deploy job then verifies the cluster exists, writes a kubeconfig, syncs the `aws-credentials` secret, applies your manifests, pins the image to this exact build, and waits up to 5 minutes for rollout. **If rollout fails it automatically runs `kubectl rollout undo`** and restores the previous revision.

Step 10 — Reach the application

```
aws eks update-kubeconfig --name ness-itbot-eks --region us-east-1
kubectl get svc -n it-support
```

If your Service is `type: LoadBalancer`, the `EXTERNAL-IP` column holds an ELB hostname — it takes a few minutes to populate. Open `http://<hostname>:8501`.

To check without a LoadBalancer:

```
kubectl port-forward -n it-support deployment/it-support-chatbot 8501:8501
```

Configuration reference

All values live in the `env:` block at the top of each workflow.

Variable	Default	Used in
<code>AWS_REGION</code>	<code>us-east-1</code>	all three

Variable	Default	Used in
STATE_BUCKET	ness-itbot-tfstate	03, 04 — must match
LOCK_TABLE	ness-itbot-tflock	03, 04 — must match
STATE_KEY	eks/terraform.tfstate	04
TF_VERSION	1.9.8	04
TF_DIR	terraform/03-eks	04
EKS_CLUSTER	ness-itbot-eks	05
DOCKERHUB_REPO	<username>/it-support-chatbot	05
K8S_DIR / K8S_NAMESPACE	k8s / it-support	05
K8S_DEPLOYMENT / K8S_CONTAINER	it-support-chatbot / chatbot	05

Troubleshooting

BucketAlreadyExists during bootstrap The name is taken globally. Append your account ID and update both 03 and 04.

Error acquiring the state lock A previous run died mid-apply. Confirm nothing is running, then `terraform force-unlock <LOCK_ID>` locally.

Backend configuration changed STATE_BUCKET or LOCK_TABLE differs between 03 and 04, or was edited after init. Make them match.

Error: Provider dependency changes or a checksum mismatch on init `.terraform.lock.hcl` is missing hashes for `linux_amd64`. A lock file generated on Windows or macOS records only that platform by default, and the Ubuntu runner then rejects it. Regenerate with every platform you build from:

```
cd terraform/03-eks
terraform providers lock \
  -platform=linux_amd64 -platform=windows_amd64 \
  -platform=darwin_amd64 -platform=darwin_arm64
```

Saved plan is stale on apply The apply job downloads the plan *before* running init, and restores the lock file the plan was built with, precisely so the provider versions match. If you reorder those steps, apply breaks.

Deploy fails: "Cluster not found" Step 8 has not completed, or EKS_CLUSTER does not match the Terraform-created name.

Rollout times out and auto-rolls-back Almost always the container failing to start. Check first:

```
kubectl describe pod -n it-support -l app=it-support-chatbot
kubectl logs -n it-support -l app=it-support-chatbot --previous
```

Common causes: invalid AWS credentials making `validate_config()` raise at startup, no Bedrock model access in the region (see below), a `K8S_CONTAINER` name mismatch, or the node being too small to schedule the pod — one `t3.small` has ~1.5 GiB allocatable against the container's 512Mi request.

ImagePullBackOff Docker Hub repo is private and the cluster has no pull secret. Make it public, or create an `imagePullSecrets` entry.

Deploy succeeds but the app errors on first message The pod has no Bedrock model access. Enable Nova Pro and Titan Embeddings in the Bedrock console under **Model access** for your region.

Tearing down

Run `04-infra-eks.yaml` with `action: destroy` (plan first, then approve). This does **not** remove the state bucket or lock table — delete those manually:

```
aws s3 rb s3://ness-itbot-tfstate --force
aws dynamodb delete-table --table-name ness-itbot-tflock --region us-east-1
```

Also delete any LoadBalancers the Service created, or they keep billing.

Security notes

- **Static keys are a deliberate trade-off** for this teaching repo. Production should use OIDC role assumption (`aws-actions/configure-aws-credentials` with `role-to-assume`) — short-lived credentials, no long-lived secrets stored.
- **The `aws-credentials` k8s Secret is base64, not encrypted.** Anyone with `get secret` in the namespace can read it. Production should use IRSA (IAM Roles for Service Accounts) so pods get credentials without a stored secret.
- **Add a `.gitignore`** before committing Terraform. At minimum: `.env`, `chroma_db/`, `.terraform/`, `*.tfstate`, `*.tfstate.*`, `*.tfvars`.
- **Rotate the access keys periodically** and delete the IAM user when finished.